
DSC 40B - Hashing

Problem 1.

Suppose a hash table with 1000 bins stores 2000 numbers. Collisions are resolved with chaining. True or False: it is possible for one of the hash table's bins to contain zero elements.

Solution: True, imagine the case where all the numbers hash to the same bin.

Problem 2.

Suppose a hash table is implemented so that it has ten bins, and that this number of bins is not increased when new elements are inserted; that is, the hash table is not allowed to "grow" or "resize". Let n be the number of elements stored in the hash table.

What is the expected time complexity of querying an element in this hash table, as a function of n ? You may assume that collisions are resolved with chaining, that the bins are linked lists, and that the hash function is "good" in that it appears to uniformly distribute elements among bins.

Solution: $\theta(n)$.

Given n elements, the expected number of elements in each bin is $n/10$ due to the assumption that the hash table doesn't resize and hash table hashes uniformly across the ten buckets. Therefore, the expected time complexity of querying an element is going to be cost of looking up the index + linear searching through the linked list, which takes $\theta(1) + \theta(n/10) = \theta(n)$.

Problem 3.

What is the expected time complexity of the code below, assuming that *numbers* is a Python *set* of size n ? Recall that Python's sets are implemented as hash tables.

```
def foo(numbers):
    count = 0
    for x in numbers:
        if -x in numbers:
            count += 1
    return count
```

Solution: $\theta(n)$

Checking if an element is in the set using the Python **in** operator takes time linear in the size of the set. Therefore, the if statement in the code takes expected $\theta(1)$ time. The for loop executes for $\theta(n)$ iterations. Hence, the total time complexity is $\theta(n)$.

Problem 4.

Given two strings **a** and **b**, determine if they are isomorphic.

Two strings **a** and **b** are isomorphic if the characters in **a** can be replaced to get **b**. They must also have the same length.

Eg. "foo" and "bar" are not isomorphic, but "egg" and "add" are. "badc" and "baba" are not isomorphic.

Describe an algorithm that determines if two strings are isomorphic.

Essentially, can the characters in **a** be mapped to the characters in **b** such that the characters in **a** can be replaced to get **b**.

Solution: The idea is that if two strings are isomorphic, then the characters in the first string can be mapped to the characters in the second string. And a character can only be mapped to one character.

We can create two dictionaries, `a2b` and `b2a` to keep track of the mapping of characters from the first string to the second string and vice versa. The keys of `a2b` are characters of `a`, and values are characters of `b`. The keys of `b2a` are characters of `b`, and values are characters of `a`.

We will loop through the characters of the strings simultaneously (they are the same length), and create a mapping for `a[i]` to `b[i]` and `b[i]` to `a[i]`.

During each iteration, whenever we see that an existing character already has a mapping, we will check if the existing mapping is the same as the current mapping to be assigned. If it is not, then the strings are not isomorphic.

If no mapping conflicts arise, then the strings are isomorphic.

```
def is_isomorphic(a: str, b: str) -> bool:
    a2b = {}
    b2a = {}
    N = len(a)
    M = len(b)

    # if they are different length isomorphism is never possible
    if N != M:
        return False

    # for each letter in A, map it to B, and vice versa
    for i in range(N):

        mapping_exists_for_a = a[i] in a2b
        mapping_exists_for_b = b[i] in b2a

        mapping_conflict_for_a = mapping_exists_for_a and a2b[a[i]] != b[i]
        mapping_conflict_for_b = mapping_exists_for_b and b2a[b[i]] != a[i]
        # if a mapping exists and they don't point to each other, it's doomed
        if mapping_conflict_for_a or mapping_conflict_for_b:
            return False

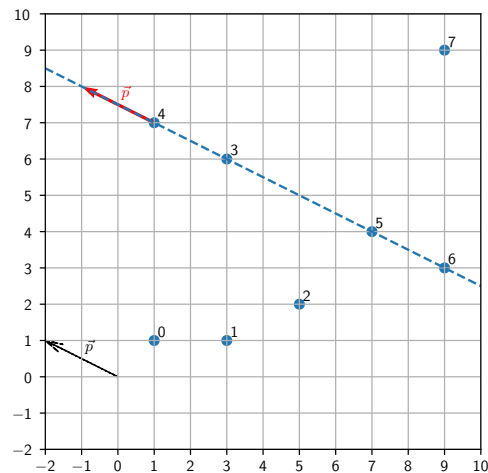
        a2b[a[i]] = b[i]
        b2a[b[i]] = a[i]

    return True
```

Problem 5.

Consider the problem of finding collinear points. Suppose you are given a set X of n points in $\mathbb{R}^2$, along with a query vector $\vec{p} \in \mathbb{R}^2$. Your goal is to calculate the maximum number of points in X that fall on the same straight line parallel to $\vec{p}$. That is, considering all of the (infinitely) many lines parallel to $\vec{p}$, find the one that contains the most points from X .

For example, in the picture below, the black points are those in X and the arrow represents $\vec{p}$. The maximum number of points in X that falls on a line parallel to $\vec{p}$ is four (those on the dashed line).



In a file named `max_points_on_line.py`, write a function `max_points_on_line(X, p)` which accepts the following arguments:

- `X`: A list of points, each point being a 2-tuple of numbers.
- `p`: A query vector represented as a 2-tuple of numbers.

Your function should return the maximum number of points in `X` that fall on the same straight line parallel to $\vec{p}$. Your answer should be an integer. If `X` is empty, return 0. If a point $\vec{x}$ in `X` is such that $\vec{x} = \vec{p}$, you should consider to be on the same line as $\vec{p}$ (even if there are no other points).

Your algorithm should take expected time that is linear in the number of points in `X`.

Example (using the points in the picture above):

```
>>> X = [
... (1, 1),
... (3, 1),
... (5, 2),
... (3, 6),
... (1, 7),
... (7, 4),
... (9, 3),
... (9, 9)
... ]
>>> p = (-2, 1)
>>> max_points_on_line(X, p)
4
```

Hint: maybe you can use a hash table (i.e., a Python dictionary)? Remember, to use hashing in solving an algorithmic problem, you often need to turn the problem into a *membership query*. For any given point x , you can calculate the along $\vec{p}$ that passes through x . How can you turn checking to see if there are other points with the same slope into a membership query?

Solution: This is an optimization problem that at first might look undoable in a finite amount of time: there are infinitely many lines along the vector $\vec{p}$, and we don't have time to try all of them.

The key insight is that all the lines parallel to $\vec{p}$ are parameterized by a single parameter: the slope is the same for all of them since they are all parallel, so their y-intercept determines them uniquely.

Therefore, checking if several points are on the same line, is equivalent to checking that the y-intercept of the line parallel to $\vec{p}$ passing through each of these points is the same.

Note: One could also use the x-intercept instead of the y-intercept, or the intersection with any other line that is not parallel to $\vec{p}$. In fact there is a slight problem with using the y-intercept: if $\vec{p}$ is vertical, then the y-intercept is not well-defined. This can be solved by either

1. Checking if $\vec{p} = (p_1, p_2)$ is vertical (i.e., $p_1 = 0$) and then using the x-intercept instead of the y-intercept.
2. Or by selecting a different line that is never parallel to $\vec{p}$, for example the line orthogonal to $\vec{p}$.

We will use approach 1 in the solution below, because it is conceptually simpler, But approach 2 is actually more robust and simpler to implement.

Therefore, the solution proceeds by making a dictionary (hash map) whose keys are y-intercepts and whose values are integers counting the number of points we've seen whose slope with $\vec{x}$ is equal to the key. We loop through all points in X and compute the y-intercept between each point and $\vec{p}$. We index into the dictionary with this y-intercept and increment the count stored.

The maximum value in the dictionary is the answer.

To compute the y-intercept of the line passing through a point $x = (x_1, x_2)$ and parallel to $\vec{p} = (p_1, p_2)$, we use the following:

note x_{inter} is the point where the line intersects the y-axis, and y_{inter} is the y-coordinate of this point (so $x_{\text{inter}} = (0, y_{\text{inter}})$).

$$x_{\text{inter}} = x + t\vec{p} \quad \text{where } t \text{ is some scalar}$$

$$\text{so, in first coordinate, } 0 = x_1 + tp_1$$

$$\text{so, } t = -\frac{x_1}{p_1}$$

$$\text{so, in second coordinate, } y_{\text{inter}} = x_2 + tp_2 = x_2 + \frac{x_1 p_2}{p_1}$$

```
def compute_y_intercept(point, vector):
    """Compute the y-intercept of the line that passes through the given point and has the given vector.

    Parameters
    -----
    point
        A 2-tuple of integers representing a point.
    vector
        A 2-tuple of integers representing a vector.

    Returns
    -----
    float

    Example
    -----

    >>> point = (1, 1)
    >>> vector = (2, 3)
```

```

>>> y_intercept(point, vector)
-1.0

"""
x1, x2 = point
p1, p2 = vector

return x2 - (x1 / p1) * p2
def max_points_on_line(X, p):
    """Compute the maximum number of points in X that fall on any line parallel to p.

    Parameters
    -----
    X
        A list of 2-tuples containing integers.
    p
        A point as a 2-tuple of integers.

    Returns
    -----
    int

    Example
    -----

    >>> X = [
    ... (1, 1),
    ... (3, 1),
    ... (5, 2),
    ... (3, 6),
    ... (1, 7),
    ... (7, 4),
    ... (9, 3),
    ... (9, 9)
    ... ]
    >>> p = (-2, 1)
    >>> max_points_on_line(X, p)
    4

    """
    # counts_by_1_intersect is a dictionary whose keys are slopes and whose values are counts
    # of the times we see a point in X on the line with p with the given y-intersect.
    # for the corner case where the slope is infinite (ie p[0] == 0),
    # we use the x coordinate of the point as the key (which is the x-intersect of the line)

    counts_by_y_intersect = {}

    for x in X:
        if p[0] == 0:
            y_intersect = x[0] # actually the x-intersect
        else:
            y_intersect = compute_y_intercept(x, p)

```

```
# Initialize the count for this slope if we haven't seen it before
if y_intersect not in counts_by_y_intersect:
    counts_by_y_intersect[y_intersect] = 0

# Update the count for this slope
counts_by_y_intersect[y_intersect] = counts_by_y_intersect[y_intersect] + 1

# the result is the maximum count of any line that passes through at least one point in X
return max(counts_by_y_intersect.values())
```